```
ROLLBACK;
```

The ALTER TABLE statement gets blocked trying to modify the trips table because the trip table is locked with exclusive access within a transaction.

Use the condition LIKE '%ALTER TABLE%' and wait_event = ‘relation“ to identify the blocking query.

The purpose of this exercise was mainly to simulate how a query or DDL modification can be blocked. How might you protect against that?

Preventing Excessive Queueing With A Lock Timeout

One protection you can add to your database is to set a lock_timeout. The purpose of this timeout is to set an upper bound on the amount of time a transaction can wait to acquire a lock. Once the timeout is exceeded, PostgreSQL will cancel the query.

This is a little different than a statement_timeout, but they both share the cancellation behavior. PostgreSQL tracks the duration of these queries and cancels them if they exceed the value you set.

The lock_timeout[6] parameter can be set database wide, within a session, or for a single SQL statement.

In the example form the last section, the lock_timeout was not set.

Try running the experiment again but this time you’ll set a lock_timeout. You’ll need two psql sessions, “psql1” and “psql2”, again each connected to the Rideshare database.

Use the statements below. The main difference compared with the earlier example is that you’re running the SET LOCAL lock_timeout = '5s'; statement which sets a lock_timeout for that transaction of 5 seconds.

sql/set_lock_timeout.sql

```
-- create psql1
-- start a transaction
BEGIN;
LOCK trips IN ACCESS EXCLUSIVE MODE;

-- create psql2
-- set a transaction level lock_timeout
BEGIN;
SET LOCAL lock_timeout = '5s';

-- Run the modification
```

6. https://postgresqlco.nf/doc/en/param/lock_timeout/